

AdaBoost

Jiri Matas and Jan Šochman

Centre for Machine Perception
Czech Technical University, Prague
<http://cmp.felk.cvut.cz>





Outline:

- ◆ AdaBoost algorithm
 - Why is of interest?
 - How it works?
 - Why it works?
- ◆ AdaBoost variants
- ◆ AdaBoost with a Totally Corrective Step (TCS)
- ◆ Experiments with a Totally Corrective Step

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



- ◆ 1990 – Boost-by-majority algorithm (Freund)
- ◆ 1995 – AdaBoost (Freund & Schapire)
- ◆ 1997 – Generalized version of AdaBoost (Schapire & Singer)
- ◆ 2001 – AdaBoost in Face Detection (Viola & Jones)

Interesting properties:

- ◆ AB is a linear classifier with all its desirable properties.
- ◆ AB output converges to the logarithm of likelihood ratio.
- ◆ AB has good generalization properties.
- ◆ AB is a feature selector with a principled strategy (minimisation of upper bound on empirical error).
- ◆ AB close to sequential decision making (it produces a sequence of gradually more complex classifiers).

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



- ◆ AdaBoost is an algorithm for constructing a "strong" classifier as linear combination

$$f(x) = \sum_{t=1}^T \alpha_t h_t(x)$$

of "simple" "weak" classifiers $h_t(x)$.

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



- ◆ AdaBoost is an algorithm for constructing a "strong" classifier as linear combination

$$f(x) = \sum_{t=1}^T \alpha_t h_t(x)$$

of "simple" "weak" classifiers $h_t(x)$.

Terminology

- ◆ $h_t(x)$... "weak" or basis classifier, hypothesis, "feature"
- ◆ $H(x) = \text{sign}(f(x))$... "strong" or final classifier/hypothesis

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



- ◆ AdaBoost is an algorithm for constructing a "strong" classifier as linear combination

$$f(x) = \sum_{t=1}^T \alpha_t h_t(x)$$

of "simple" "weak" classifiers $h_t(x)$.

Terminology

- ◆ $h_t(x)$... "weak" or basis classifier, hypothesis, "feature"
- ◆ $H(x) = \text{sign}(f(x))$... "strong" or final classifier/hypothesis

Comments

- ◆ The $h_t(x)$'s can be thought of as features.
- ◆ Often (typically) the set $\mathcal{H} = \{h(x)\}$ is infinite.

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



Given: $(x_1, y_1), \dots, (x_m, y_m)$; $x_i \in \mathcal{X}, y_i \in \{-1, 1\}$

Initialize weights $D_1(i) = 1/m$

For $t = 1, \dots, T$:

1. (Call *WeakLearn*), which returns the weak classifier $h_t : \mathcal{X} \rightarrow \{-1, 1\}$ with minimum error w.r.t. distribution D_t ;
2. Choose $\alpha_t \in R$,
3. Update

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

where Z_t is a normalization factor chosen so that D_{t+1} is a distribution

Output the strong classifier:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



Given: $(x_1, y_1), \dots, (x_m, y_m)$; $x_i \in \mathcal{X}, y_i \in \{-1, 1\}$

Initialize weights $D_1(i) = 1/m$

For $t = 1, \dots, T$:

1. (Call *WeakLearn*), which returns the weak classifier $h_t : \mathcal{X} \rightarrow \{-1, 1\}$ with minimum error w.r.t. distribution D_t ;
2. Choose $\alpha_t \in R$,
3. Update

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

where Z_t is a normalization factor chosen so that D_{t+1} is a distribution

Output the strong classifier:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

Comments

- ◆ The computational complexity of selecting h_t is independent of t .
- ◆ All information about previously selected “features” is captured in D_t !

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



Loop step: Call *WeakLearn*, given distribution D_t ;
returns weak classifier $h_t : \mathcal{X} \rightarrow \{-1, 1\}$ from $\mathcal{H} = \{h(x)\}$

- ◆ Select a weak classifier with the smallest weighted error
$$h_t = \arg \min_{h_j \in \mathcal{H}} \epsilon_j = \sum_{i=1}^m D_t(i) [y_i \neq h_j(x_i)]$$
- ◆ Prerequisite: $\epsilon_t < 1/2$ (otherwise stop)
- ◆ *WeakLearn* examples:
 - Decision tree builder, perceptron learning rule – $\mathcal{H}$ *infinite*
 - Selecting the best one from given *finite* set $\mathcal{H}$

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



Loop step: Call *WeakLearn*, given distribution D_t ;
returns weak classifier $h_t : \mathcal{X} \rightarrow \{-1, 1\}$ from $\mathcal{H} = \{h(x)\}$

- ◆ Select a weak classifier with the smallest weighted error

$$h_t = \arg \min_{h_j \in \mathcal{H}} \epsilon_j = \sum_{i=1}^m D_t(i) [y_i \neq h_j(x_i)]$$
- ◆ Prerequisite: $\epsilon_t < 1/2$ (otherwise stop)
- ◆ *WeakLearn* examples:
 - Decision tree builder, perceptron learning rule – $\mathcal{H}$ *infinite*
 - Selecting the best one from given *finite* set $\mathcal{H}$

Demonstration example

Weak classifier = perceptron

$$\bullet \sim N(0, 1) \quad \bullet \sim \frac{1}{r\sqrt{8\pi^3}} e^{-1/2(r-4)^2}$$

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



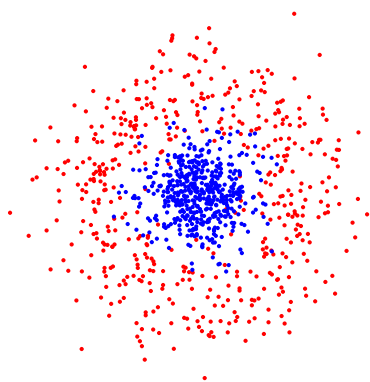
Loop step: Call *WeakLearn*, given distribution D_t ;
returns weak classifier $h_t : \mathcal{X} \rightarrow \{-1, 1\}$ from $\mathcal{H} = \{h(x)\}$

- ◆ Select a weak classifier with the smallest weighted error

$$h_t = \arg \min_{h_j \in \mathcal{H}} \epsilon_j = \sum_{i=1}^m D_t(i) [y_i \neq h_j(x_i)]$$
- ◆ Prerequisite: $\epsilon_t < 1/2$ (otherwise stop)
- ◆ *WeakLearn* examples:
 - Decision tree builder, perceptron learning rule – $\mathcal{H}$ *infinite*
 - Selecting the best one from given *finite* set $\mathcal{H}$

Demonstration example

Training set



Weak classifier = perceptron

$$\bullet \sim N(0, 1) \quad \bullet \sim \frac{1}{r\sqrt{8\pi^3}} e^{-1/2(r-4)^2}$$

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

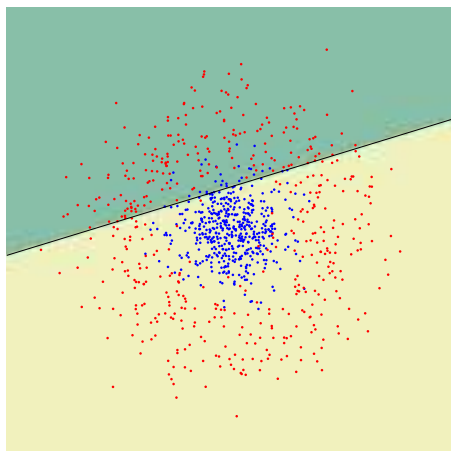
Loop step: Call *WeakLearn*, given distribution D_t ;
returns weak classifier $h_t : \mathcal{X} \rightarrow \{-1, 1\}$ from $\mathcal{H} = \{h(x)\}$

- ◆ Select a weak classifier with the smallest weighted error

$$h_t = \arg \min_{h_j \in \mathcal{H}} \epsilon_j = \sum_{i=1}^m D_t(i) [y_i \neq h_j(x_i)]$$
- ◆ Prerequisite: $\epsilon_t < 1/2$ (otherwise stop)
- ◆ *WeakLearn* examples:
 - Decision tree builder, perceptron learning rule – $\mathcal{H}$ *infinite*
 - Selecting the best one from given *finite* set $\mathcal{H}$

Demonstration example

Training set



Weak classifier = perceptron

• $\sim N(0, 1)$

• $\sim \frac{1}{r\sqrt{8\pi^3}} e^{-1/2(r-4)^2}$

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



- ◆ The main objective is to minimize $\varepsilon_{tr} = \frac{1}{m} |\{i : H(x_i) \neq y_i\}|$
- ◆ It can be upper bounded by $\varepsilon_{tr}(H) \leq \prod_{t=1}^T Z_t$

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



- ◆ The main objective is to minimize $\varepsilon_{tr} = \frac{1}{m} |\{i : H(x_i) \neq y_i\}|$
- ◆ It can be upper bounded by $\varepsilon_{tr}(H) \leq \prod_{t=1}^T Z_t$

How to set α_t ?

- ◆ Select α_t to greedily minimize $Z_t(\alpha)$ in each step
- ◆ $Z_t(\alpha)$ is convex differentiable function with one extremum
 - $\Rightarrow h_t(x) \in \{-1, 1\}$ then optimal $\alpha_t = \frac{1}{2} \log\left(\frac{1+r_t}{1-r_t}\right)$
 - where $r_t = \sum_{i=1}^m D_t(i) h_t(x_i) y_i$
- ◆ $Z_t = 2\sqrt{\epsilon_t(1 - \epsilon_t)} \leq 1$ for optimal α_t
 - $\Rightarrow$ Justification of selection of h_t according to ϵ_t

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



- ◆ The main objective is to minimize $\varepsilon_{tr} = \frac{1}{m} |\{i : H(x_i) \neq y_i\}|$
- ◆ It can be upper bounded by $\varepsilon_{tr}(H) \leq \prod_{t=1}^T Z_t$

How to set α_t ?

- ◆ Select α_t to greedily minimize $Z_t(\alpha)$ in each step
- ◆ $Z_t(\alpha)$ is convex differentiable function with one extremum
 $\Rightarrow h_t(x) \in \{-1, 1\}$ then optimal $\alpha_t = \frac{1}{2} \log\left(\frac{1+r_t}{1-r_t}\right)$
 where $r_t = \sum_{i=1}^m D_t(i) h_t(x_i) y_i$
- ◆ $Z_t = 2\sqrt{\epsilon_t(1 - \epsilon_t)} \leq 1$ for optimal α_t
 $\Rightarrow$ Justification of selection of h_t according to ϵ_t

Comments

- ◆ The process of selecting α_t and $h_t(x)$ can be interpreted as a single optimization step minimising the upper bound on the empirical error. Improvement of the bound is guaranteed, provided that $\epsilon_t < 1/2$.
- ◆ The process can be interpreted as a component-wise local optimization (Gauss-Southwell iteration) in the (possibly infinite dimensional!) space of $\bar{\alpha} = (\alpha_1, \alpha_2, \dots)$ starting from $\bar{\alpha}_0 = (0, 0, \dots)$.

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



Effect on the training set

Reweighting formula:

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t} = \frac{\exp(-y_i \sum_{q=1}^t \alpha_q h_q(x_i))}{m \prod_{q=1}^t Z_q}$$

$$\exp(-\alpha_t y_i h_t(x_i)) \begin{cases} < 1, & y_i = h_t(x_i) \\ > 1, & y_i \neq h_t(x_i) \end{cases}$$

⇒ Increase (decrease) weight of wrongly (correctly) classified examples. The weight is the upper bound on the error of a given example!

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



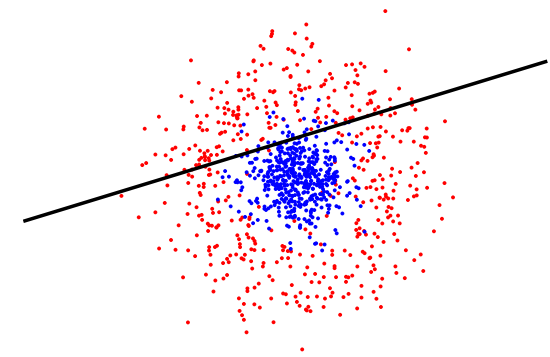
Effect on the training set

Reweighting formula:

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t} = \frac{\exp(-y_i \sum_{q=1}^t \alpha_q h_q(x_i))}{m \prod_{q=1}^t Z_q}$$

$$\exp(-\alpha_t y_i h_t(x_i)) \begin{cases} < 1, & y_i = h_t(x_i) \\ > 1, & y_i \neq h_t(x_i) \end{cases}$$

⇒ Increase (decrease) weight of wrongly (correctly) classified examples. The weight is the upper bound on the error of a given example!



1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



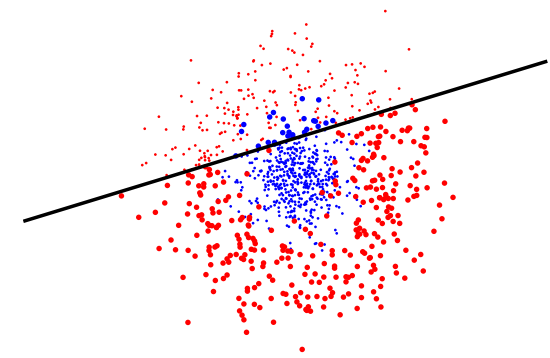
Effect on the training set

Reweighting formula:

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t} = \frac{\exp(-y_i \sum_{q=1}^t \alpha_q h_q(x_i))}{m \prod_{q=1}^t Z_q}$$

$$\exp(-\alpha_t y_i h_t(x_i)) \begin{cases} < 1, & y_i = h_t(x_i) \\ > 1, & y_i \neq h_t(x_i) \end{cases}$$

⇒ Increase (decrease) weight of wrongly (correctly) classified examples. The weight is the upper bound on the error of a given example!



1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



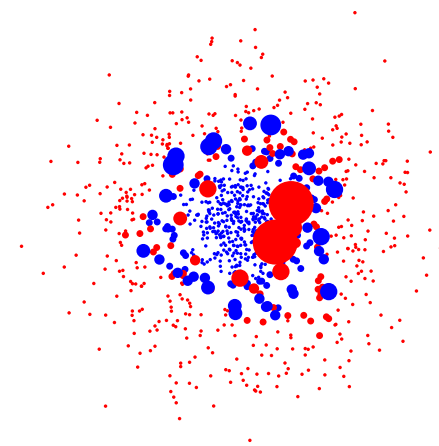
Effect on the training set

Reweighting formula:

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t} = \frac{\exp(-y_i \sum_{q=1}^t \alpha_q h_q(x_i))}{m \prod_{q=1}^t Z_q}$$

$$\exp(-\alpha_t y_i h_t(x_i)) \begin{cases} < 1, & y_i = h_t(x_i) \\ > 1, & y_i \neq h_t(x_i) \end{cases}$$

⇒ Increase (decrease) weight of wrongly (correctly) classified examples. The weight is the upper bound on the error of a given example!



1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



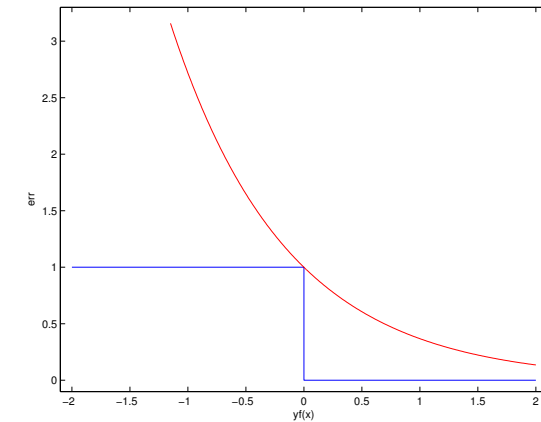
Effect on the training set

Reweighting formula:

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t} = \frac{\exp(-y_i \sum_{q=1}^t \alpha_q h_q(x_i))}{m \prod_{q=1}^t Z_q}$$

$$\exp(-\alpha_t y_i h_t(x_i)) \begin{cases} < 1, & y_i = h_t(x_i) \\ > 1, & y_i \neq h_t(x_i) \end{cases}$$

⇒ Increase (decrease) weight of wrongly (correctly) classified examples. The weight is the upper bound on the error of a given example!



1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



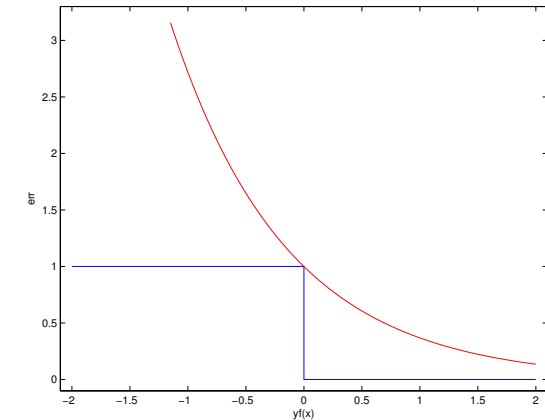
Effect on the training set

Reweighting formula:

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t} = \frac{\exp(-y_i \sum_{q=1}^t \alpha_q h_q(x_i))}{m \prod_{q=1}^t Z_q}$$

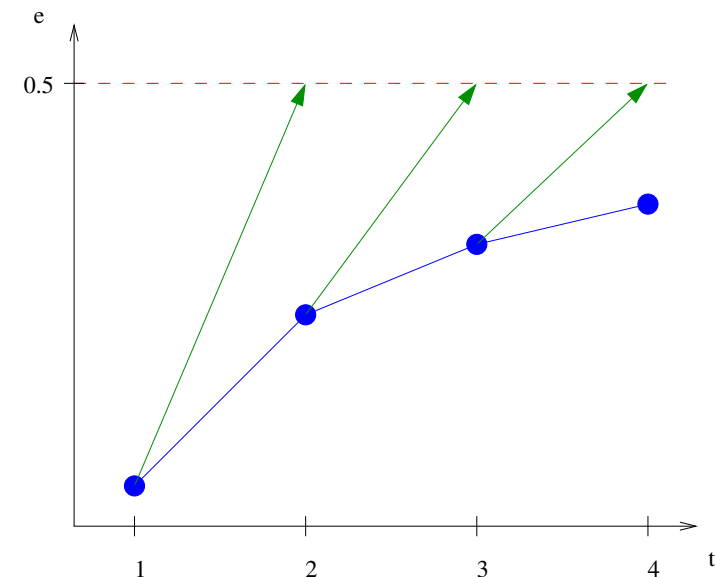
$$\exp(-\alpha_t y_i h_t(x_i)) \begin{cases} < 1, & y_i = h_t(x_i) \\ > 1, & y_i \neq h_t(x_i) \end{cases}$$

⇒ Increase (decrease) weight of wrongly (correctly) classified examples. The weight is the upper bound on the error of a given example!



Effect on h_t

- ◆ α_t minimize $Z_t \Rightarrow \sum_{i: h_t(x_i)=y_i} D_{t+1}(i) = \sum_{i: h_t(x_i) \neq y_i} D_{t+1}(i)$
- ◆ Error of h_t on D_{t+1} is $1/2$
- ◆ Next weak classifier is the most “independent” one



1

2

3

4

5

6

7

8

9

10

11

12

13

14

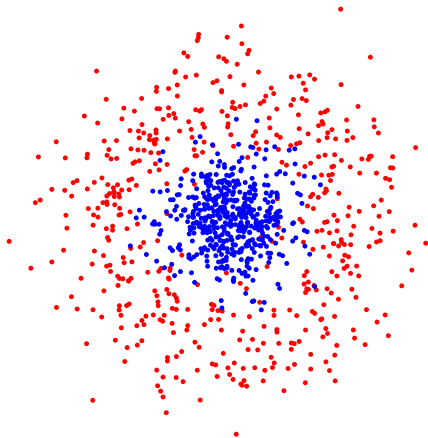
15

16

Summary of the Algorithm										m p	
										1	
										2	
										3	
										4	
										5	
										6	
										7	
										8	
										9	
										10	
										11	
										12	
										13	
										14	
										15	
										16	



Initialization...



1

2

3

4

5

6

7

8

9

10

11

12

13

14

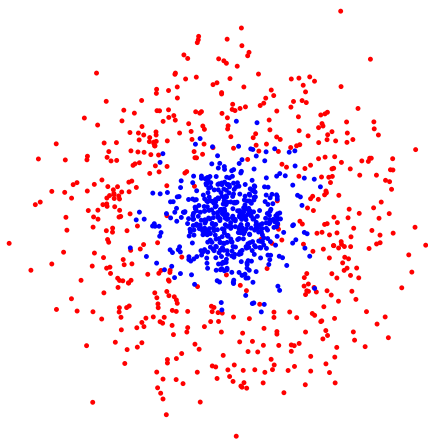
15

16



Initialization...

For $t = 1, \dots, T$:



1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

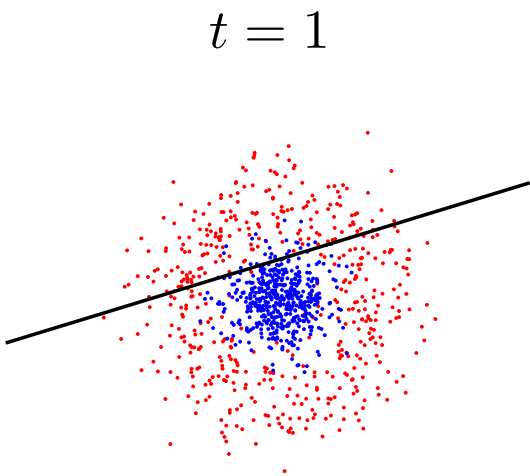
16



Initialization...

For $t = 1, \dots, T$:

♦ Find $h_t = \arg \min_{h_j \in \mathcal{H}} \epsilon_j = \sum_{i=1}^m D_t(i)[y_i \neq h_j(x_i)]$



1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16

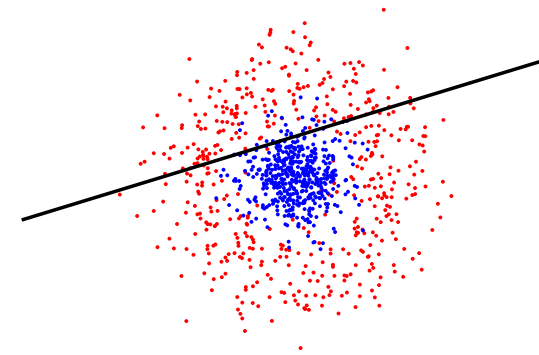


Initialization...

For $t = 1, \dots, T$:

- ◆ Find $h_t = \arg \min_{h_j \in \mathcal{H}} \epsilon_j = \sum_{i=1}^m D_t(i) [y_i \neq h_j(x_i)]$
- ◆ If $\epsilon_t \geq 1/2$ then stop

$t = 1$



1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

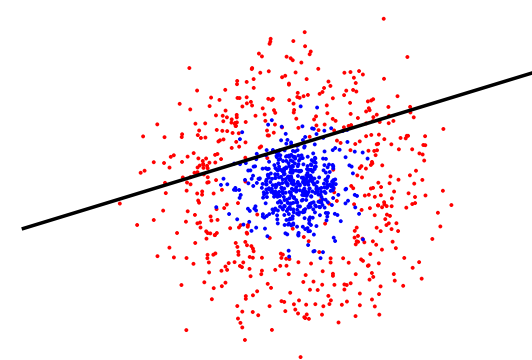


Initialization...

For $t = 1, \dots, T$:

- ◆ Find $h_t = \arg \min_{h_j \in \mathcal{H}} \epsilon_j = \sum_{i=1}^m D_t(i) [y_i \neq h_j(x_i)]$
- ◆ If $\epsilon_t \geq 1/2$ then stop
- ◆ Set $\alpha_t = \frac{1}{2} \log\left(\frac{1+r_t}{1-r_t}\right)$

$t = 1$



1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



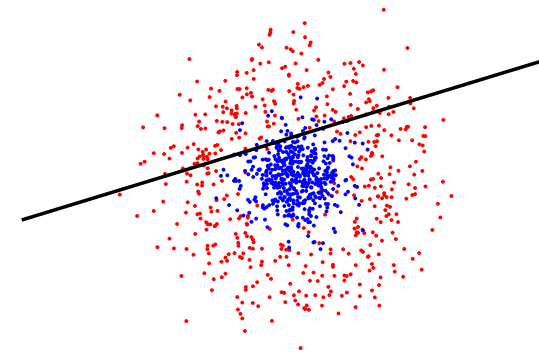
Initialization...

For $t = 1, \dots, T$:

- ◆ Find $h_t = \arg \min_{h_j \in \mathcal{H}} \epsilon_j = \sum_{i=1}^m D_t(i) [y_i \neq h_j(x_i)]$
- ◆ If $\epsilon_t \geq 1/2$ then stop
- ◆ Set $\alpha_t = \frac{1}{2} \log\left(\frac{1+r_t}{1-r_t}\right)$
- ◆ Update

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

$t = 1$



1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



Initialization...

For $t = 1, \dots, T$:

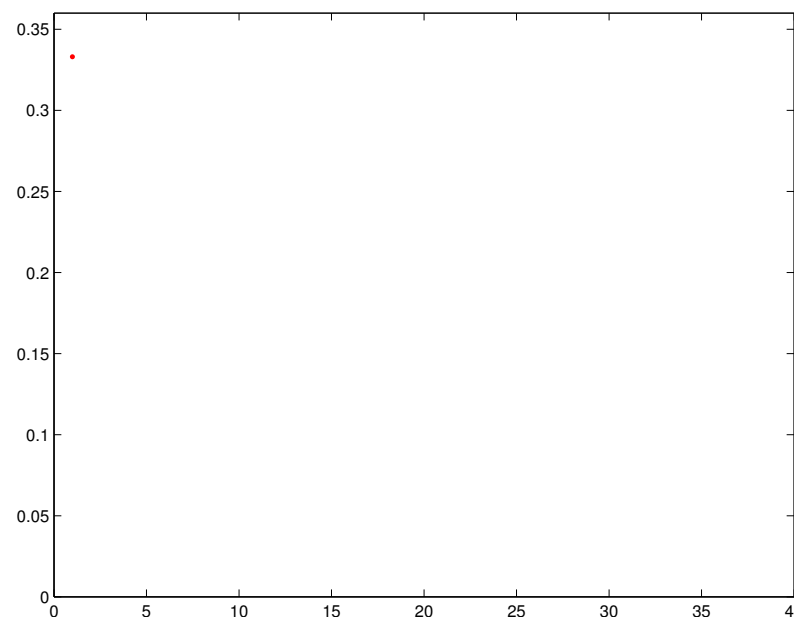
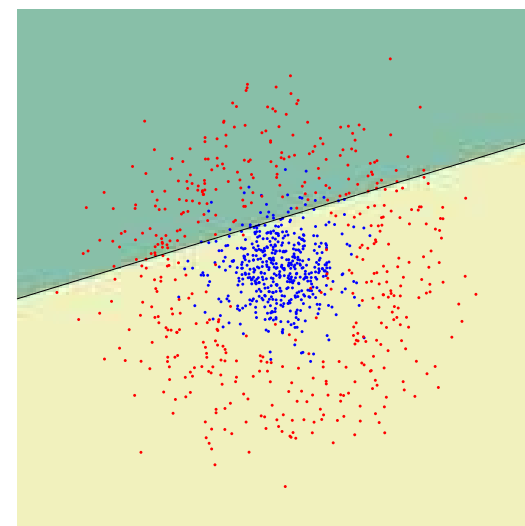
- ◆ Find $h_t = \arg \min_{h_j \in \mathcal{H}} \epsilon_j = \sum_{i=1}^m D_t(i) [y_i \neq h_j(x_i)]$
- ◆ If $\epsilon_t \geq 1/2$ then stop
- ◆ Set $\alpha_t = \frac{1}{2} \log\left(\frac{1+r_t}{1-r_t}\right)$
- ◆ Update

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

Output the final classifier:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

$t = 1$



1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



Initialization...

For $t = 1, \dots, T$:

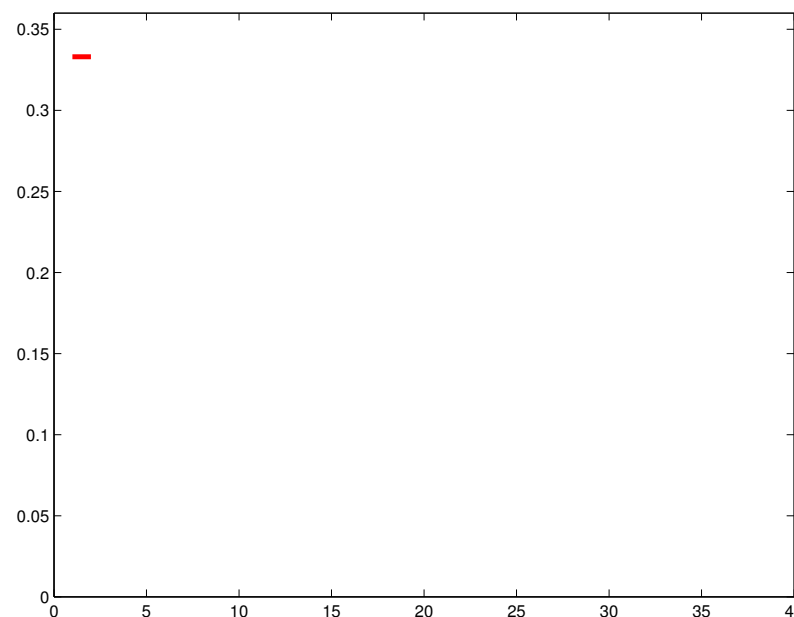
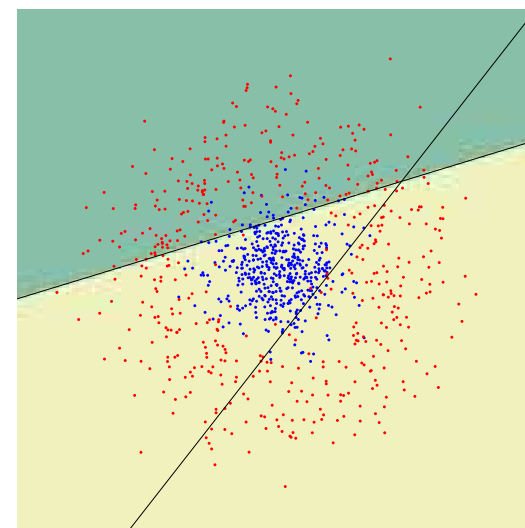
- ◆ Find $h_t = \arg \min_{h_j \in \mathcal{H}} \epsilon_j = \sum_{i=1}^m D_t(i) [y_i \neq h_j(x_i)]$
- ◆ If $\epsilon_t \geq 1/2$ then stop
- ◆ Set $\alpha_t = \frac{1}{2} \log\left(\frac{1+r_t}{1-r_t}\right)$
- ◆ Update

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

Output the final classifier:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

$t = 2$



1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



Initialization...

For $t = 1, \dots, T$:

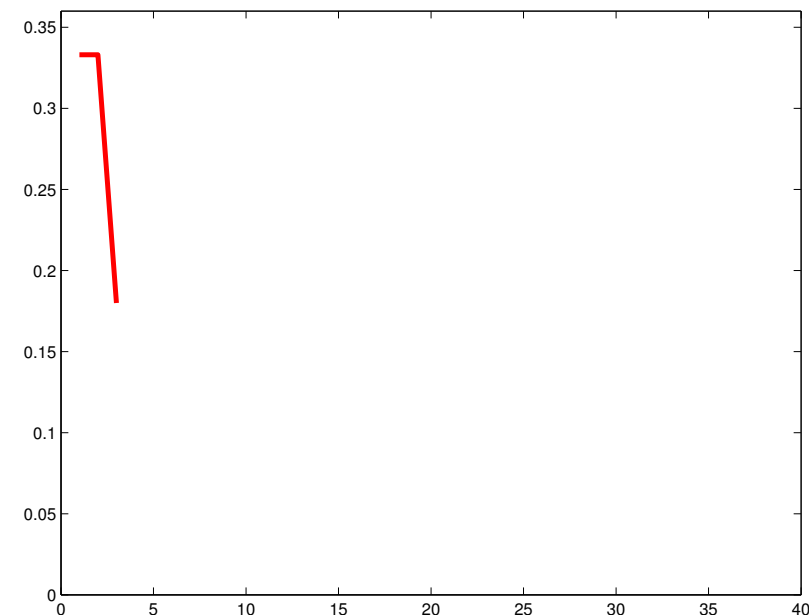
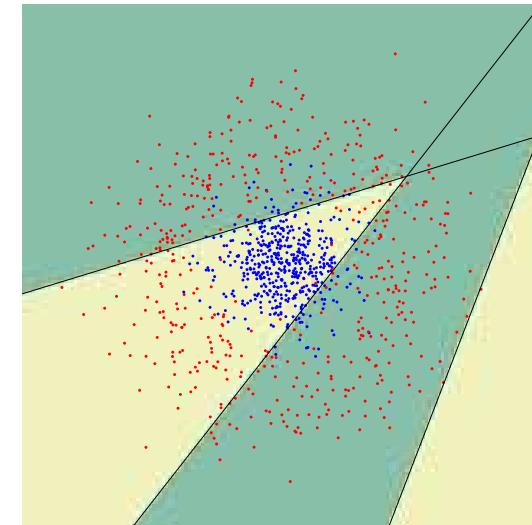
- ◆ Find $h_t = \arg \min_{h_j \in \mathcal{H}} \epsilon_j = \sum_{i=1}^m D_t(i) [y_i \neq h_j(x_i)]$
- ◆ If $\epsilon_t \geq 1/2$ then stop
- ◆ Set $\alpha_t = \frac{1}{2} \log\left(\frac{1+r_t}{1-r_t}\right)$
- ◆ Update

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

Output the final classifier:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

$t = 3$



1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



Initialization...

For $t = 1, \dots, T$:

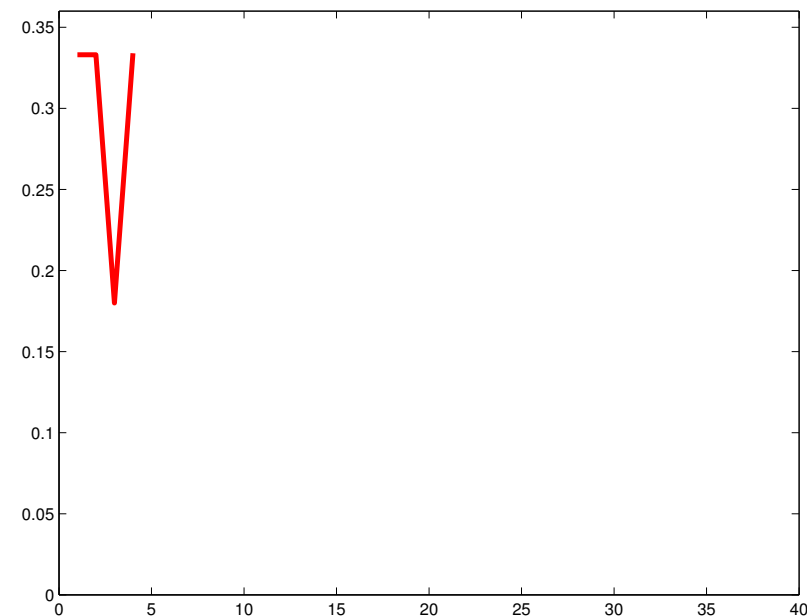
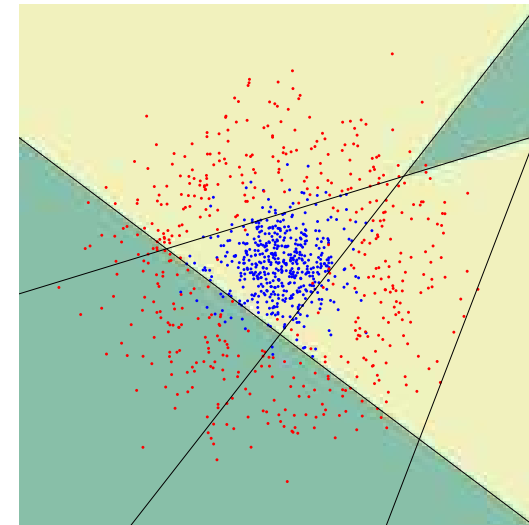
- ◆ Find $h_t = \arg \min_{h_j \in \mathcal{H}} \epsilon_j = \sum_{i=1}^m D_t(i) [y_i \neq h_j(x_i)]$
- ◆ If $\epsilon_t \geq 1/2$ then stop
- ◆ Set $\alpha_t = \frac{1}{2} \log\left(\frac{1+r_t}{1-r_t}\right)$
- ◆ Update

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

Output the final classifier:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

$t = 4$



1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



Initialization...

For $t = 1, \dots, T$:

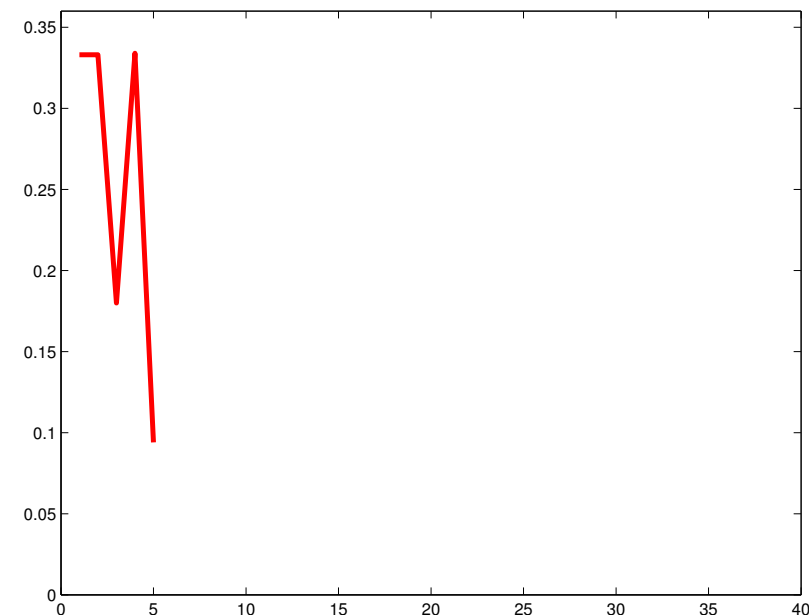
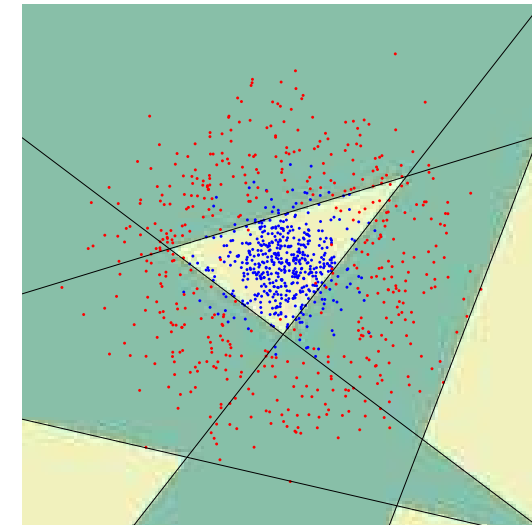
- ◆ Find $h_t = \arg \min_{h_j \in \mathcal{H}} \epsilon_j = \sum_{i=1}^m D_t(i) [y_i \neq h_j(x_i)]$
- ◆ If $\epsilon_t \geq 1/2$ then stop
- ◆ Set $\alpha_t = \frac{1}{2} \log\left(\frac{1+r_t}{1-r_t}\right)$
- ◆ Update

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

Output the final classifier:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

$t = 5$



1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



Initialization...

For $t = 1, \dots, T$:

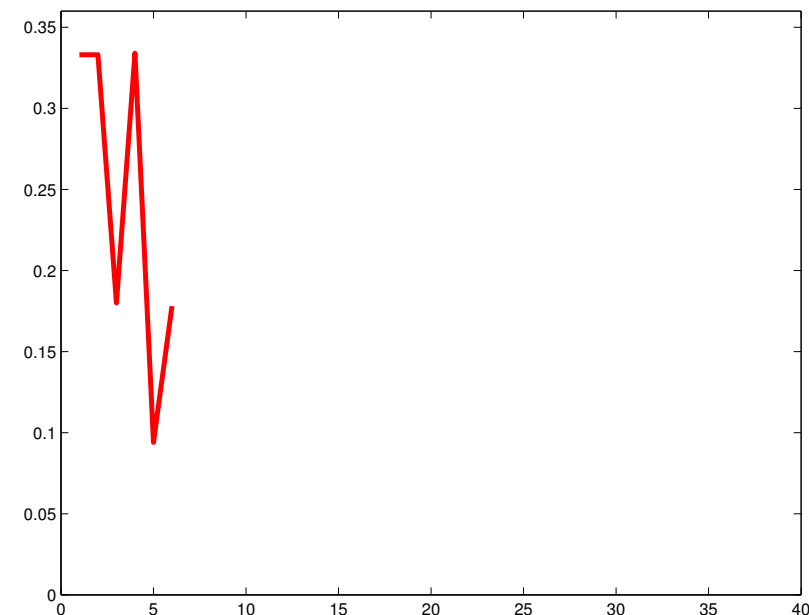
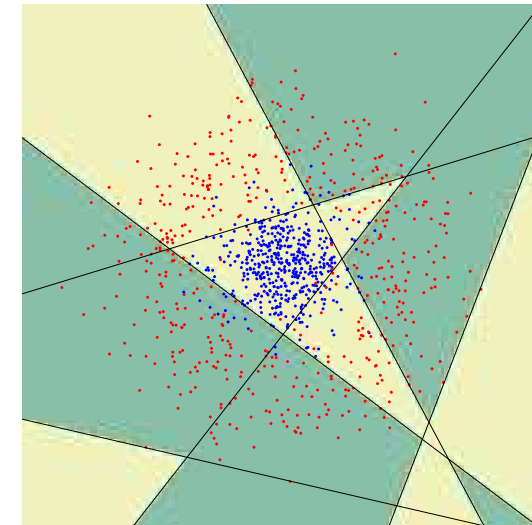
- ◆ Find $h_t = \arg \min_{h_j \in \mathcal{H}} \epsilon_j = \sum_{i=1}^m D_t(i) [y_i \neq h_j(x_i)]$
- ◆ If $\epsilon_t \geq 1/2$ then stop
- ◆ Set $\alpha_t = \frac{1}{2} \log\left(\frac{1+r_t}{1-r_t}\right)$
- ◆ Update

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

Output the final classifier:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

$t = 6$



1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



Initialization...

For $t = 1, \dots, T$:

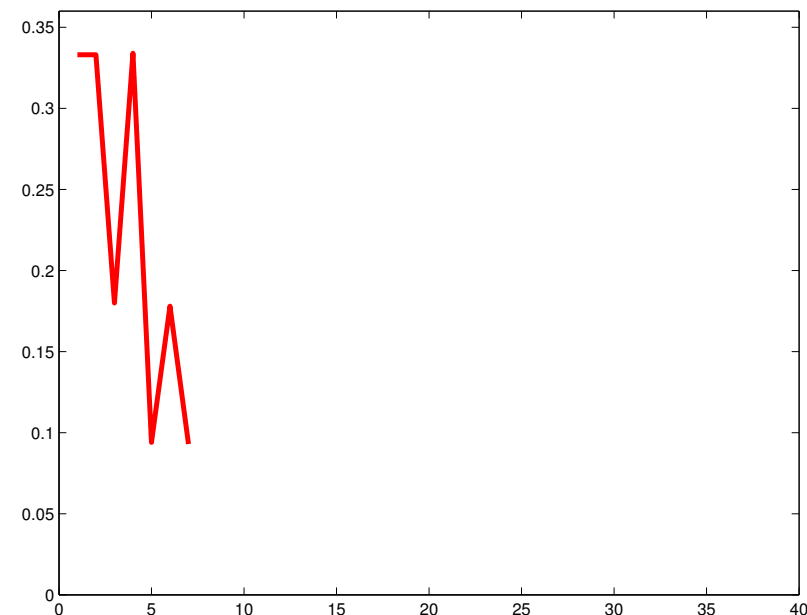
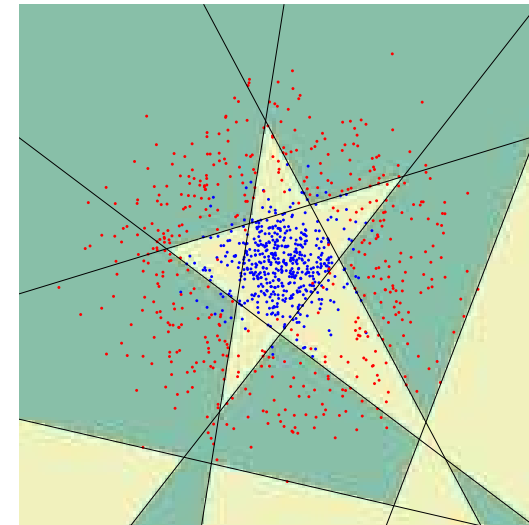
- ◆ Find $h_t = \arg \min_{h_j \in \mathcal{H}} \epsilon_j = \sum_{i=1}^m D_t(i) [y_i \neq h_j(x_i)]$
- ◆ If $\epsilon_t \geq 1/2$ then stop
- ◆ Set $\alpha_t = \frac{1}{2} \log\left(\frac{1+r_t}{1-r_t}\right)$
- ◆ Update

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

Output the final classifier:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

$t = 7$



1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



Initialization...

For $t = 1, \dots, T$:

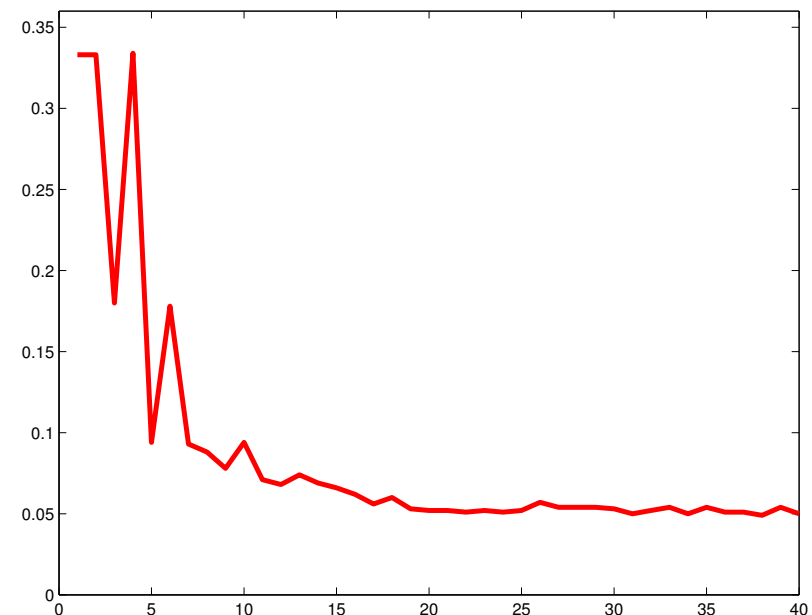
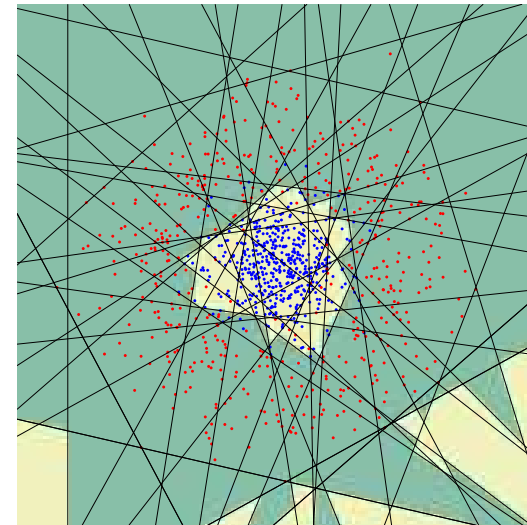
- ◆ Find $h_t = \arg \min_{h_j \in \mathcal{H}} \epsilon_j = \sum_{i=1}^m D_t(i) [y_i \neq h_j(x_i)]$
- ◆ If $\epsilon_t \geq 1/2$ then stop
- ◆ Set $\alpha_t = \frac{1}{2} \log\left(\frac{1+r_t}{1-r_t}\right)$
- ◆ Update

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

Output the final classifier:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

$t = 40$



1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



Margins in SVM

$$\max \min_{(x,y) \in S} \frac{y(\vec{\alpha} \cdot \vec{h}(x))}{\|\vec{\alpha}\|_2}$$

Margins in AdaBoost

$$\max \min_{(x,y) \in S} \frac{y(\vec{\alpha} \cdot \vec{h}(x))}{\|\vec{\alpha}\|_1}$$

Maximizing margins in AdaBoost

$$P_S[yf(x) \leq \theta] \leq 2^T \prod_{t=1}^T \sqrt{\epsilon_t^{1-\theta} (1 - \epsilon_t)^{1+\theta}} \quad \text{where } f(x) = \frac{\vec{\alpha} \cdot \vec{h}(x)}{\|\vec{\alpha}\|_1}$$

Upper bounds based on margin

$$P_{\mathcal{D}}[yf(x) \leq 0] \leq P_S[yf(x) \leq \theta] + \mathcal{O} \left(\frac{1}{\sqrt{m}} \left(\frac{d \log^2(m/d)}{\theta^2} + \log(1/\delta) \right)^{1/2} \right)$$

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



Freund & Schapire 1995

- ◆ Discrete ($h : \mathcal{X} \rightarrow \{0, 1\}$)
- ◆ Multiclass AdaBoost.M1 ($h : \mathcal{X} \rightarrow \{0, 1, \dots, k\}$)
- ◆ Multiclass AdaBoost.M2 ($h : \mathcal{X} \rightarrow [0, 1]^k$)
- ◆ Real valued AdaBoost.R ($Y = [0, 1], h : \mathcal{X} \rightarrow [0, 1]$)

Schapire & Singer 1997

- ◆ Confidence rated prediction ($h : \mathcal{X} \rightarrow R$, two-class)
- ◆ Multilabel AdaBoost.MR, AdaBoost.MH (different formulation of minimized loss)

... Many other modifications since then (Totally Corrective AB, Cascaded AB)

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16



Advantages

- ◆ Very simple to implement
- ◆ Feature selection on very large sets of features
- ◆ Fairly good generalization

Disadvantages

- ◆ Suboptimal solution for $\bar{\alpha}$
- ◆ Can overfit in presence of noise

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16